AIM2 Preparation
Week 1 - Day 2: Lists & Dictionaries

5-Minute Review Notes | Python Data Structures & Student Gradebook

Day 2 Goal

Learn how to store, access, update, and process collections of related data using Python lists and dictionaries.

1. Python Lists

```
students = ["Krishna", "Rahul", "Priya", "Amit"]
```

A list stores multiple values in an ordered collection. Lists are mutable, meaning their contents can be changed.

2. List Indexing

```
students[0] # Krishna
students[1] # Rahul
students[-1] # Amit
```

Python indexing starts at 0. Negative indexes count backward from the end of the list.

3. Updating List Elements

```
students[1] = "Ravi"
```

Because lists are mutable, an existing element can be replaced using its index.

4. Common List Methods

Method	Purpose
append(value)	Add one value to the end.
insert(index, value)	Add a value at a specific position.
remove(value)	Remove the first matching value.
pop()	Remove and return an item (last by default).
len(list)	Return the number of elements.

```
students.append("Neha")
students.insert(1, "Sara")
students.remove("Amit")
removed = students.pop()
print(len(students))
```

5. Dictionaries

```
student = {
  "Name": "Krishna",
  "Math": 85,
  "Python": 92
}
```

A dictionary stores data as key-value pairs. Keys describe what each value represents.

6. Accessing and Updating Dictionary Data

```
student["Name"]
student["Math"]
```

```
student["Math"] = 90
student["AI"] = 88
```

Use a key instead of a numeric position. Assigning to an existing key updates it; assigning a new key adds a new entry.

7. Dictionary Keys and Values

```
student.keys()
student.values()
student.items()
```

- keys() returns the dictionary keys.
- values() returns the stored values.
- items() returns key-value pairs.

8. List of Dictionaries

```
students = [
  {"Name": "Krishna", "Math": 85, "Python": 92},
  {"Name": "Rahul", "Math": 75, "Python": 80},
  {"Name": "Priya", "Math": 91, "Python": 95}
]
```

This structure is useful for tabular-style data: the list stores many records, and each dictionary represents one student.

9. Iterating Through Student Records

```
for student in students:
    print(student["Name"])
    print(student["Math"])
```

The loop processes one dictionary (one student record) at a time.

10. Basic Data Processing

```
for student in students:
    total = student["Math"] + student["Python"] + student["AI"]
    average = total / 3
```

Combining lists, dictionaries, loops, and arithmetic lets us turn stored data into useful information.

List vs Dictionary

List	Dictionary
Ordered collection of values	Key-value collection
Access using index	Access using key
students[0]	student["Name"]
Best for sequences/collections	Best for structured records

Day 2 Mini-Project - Student Gradebook

The Student Gradebook combined a list of dictionaries with basic data processing.

- Stored multiple student records.
- Stored subject scores for each student.
- Calculated each student's total score.

- Calculated each student's average score.
- Determined pass/fail status.
- Printed formatted results for every student.

Quick Syntax Cheat Sheet

```
my_list = [value1, value2]
my_list[0]
my_list[-1]
my_list.append(value)
my_list.insert(index, value)
my_list.remove(value)
my_list.pop()
len(my_list)

student = {"Name": "Krishna", "Math": 85}
student["Name"]
student["Math"] = 90
student.keys()
student.values()
student.items()

for student in students:
    print(student["Name"])
```

Key Takeaways

- Lists store multiple ordered values and use indexes.
- Negative indexing accesses values from the end of a list.
- Lists are mutable and support append, insert, remove, and pop.
- Dictionaries store structured information using key-value pairs.
- A list of dictionaries is useful for storing multiple structured records.
- Loops can process each record and calculate totals, averages, and results.
- These structures prepare you for Pandas DataFrames, where rows and columns represent similar structured data.

Progress

Week 1 Day 2 - Lists & Dictionaries: COMPLETE
Mini-project: Student Gradebook.